

## Cours 9 — Tableaux et chaînes de caractères

Semaine 9 ■ Manuel Hachette ch. 4 (§ 4.3, 4.4) ■ 2 × 1 h

### Objectifs

- Créer, lire et **modifier** un tableau (liste Python)
- Parcourir un tableau par éléments et par indices
- Manipuler les chaînes comme des séquences ; construire une liste en compréhension

## 1. Le tableau (liste Python)

Un **tableau** est une séquence **modifiable** de valeurs, généralement de même type :

```
notes = [12, 15, 9, 17]
notes[0]          # 12
notes[1] = 16      # modification : [12, 16, 9, 17]
len(notes)         # 4
```

Contrairement au tuple, la liste est **mutable** : on peut modifier, ajouter, retirer.

```
notes.append(11)    # ajoute à la fin
[0] * 5             # [0, 0, 0, 0, 0] : création par répétition
```

## 2. Les deux parcours

```
# par éléments : on lit les valeurs
for note in notes:
    print(note)

# par indices : on peut MODIFIER les cases
for i in range(len(notes)):
    notes[i] = notes[i] + 1
```

Pour **lire**, le parcours par éléments suffit et il est plus lisible. Pour **modifier** les cases, il faut les indices. (Modifier la variable de boucle **note** ne modifie pas le tableau !)

## 3. Les chaînes, des séquences (presque) comme les autres

```
mot = "python"
mot[0]          # 'p'
len(mot)        # 6
for c in mot:   # parcours par caractères
    ...
"y" in mot      # True : test d'appartenance (marche aussi sur les listes)
```

Mais la chaîne est **immuable** comme le tuple : `mot[0] = "P"` est interdit — on construit une nouvelle chaîne (TP 6 : accumulation).

**Tranches** (*slices*), sur chaînes et listes :

```
mot[1:4]      # 'yth' (indices 1, 2, 3 : borne droite exclue)
mot[:2]       # 'py'
mot[2:]       # 'thon'
```

## 4. Construire une liste en compréhension

Le motif « partir de [] et accumuler avec `append` » est si fréquent que Python offre une écriture compacte, au programme de NSI :

```
carres = [n * n for n in range(10)]
pairs  = [n for n in range(20) if n % 2 == 0]
codes  = [ord(c) for c in "NSI"]           # [78, 83, 73]
```

Se lit : « la liste des  $n^2$  pour  $n$  allant de 0 à 9 ». La version avec `if` filtre les éléments retenus.

### À retenir

- Liste = séquence **modifiable** ; `append` ajoute ; `[0] * n` initialise.
- Lire  $\rightarrow$  `for x in t` ; modifier  $\rightarrow$  `for i in range(len(t))`.
- Chaînes : mêmes indices, tranches et parcours, mais **immuables**.
- Compréhension : `[expression for x in sequence if condition]`.